Module7_Lab1

October 16, 2026

[1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('tips.csv')

X = df[['total_bill']]
y = df['tip']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (170, 1)
X_test shape: (74, 1)
y_train shape: (170,)
y_test shape: (74,)
```

In 2–3 sentences, explain why we grade on data the model never saw.

For one, as we learned in the lecture, this is to prevent memorization versus actually learning. For instance, if we were shown the exact exam the night before, we could simply memoriza the answers instead of actually learning the materials that we need to learn.

[2]:
```
from sklearn.linear_model import LinearRegression
df = pd.read_csv('tips.csv')

X = df[['total_bill']]
y = df['tip']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

slope = model.coef_[0]
```

```
intercept = model.intercept_

print(model.coef_, model.intercept_)
```

```
[0.10889371] 0.8769576391532707
```

[3]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('tips.csv')
X = df[['total_bill']]
y = df['tip']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

X_line = pd.DataFrame({'total_bill': sorted(df['total_bill'])})
y_pred = model.predict(X_line)

plt.figure
plt.scatter(df['total_bill'], df['tip']
plt.plot(X_line, y_pred, color='red', linewidth=2, label=f'Fitted Line: y =␣
 ↪{model.coef_[0]:.2f}x + {model.intercept_:.2f}')

plt.title('Total Bill vs. Tip with Fitted Linear Regression Line')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.legend
plt.grid

plt.show()
```

```
  Cell In[3], line 18
    plt.scatter(df['total_bill'], df['tip']
               ^
SyntaxError: '(' was never closed
```

[4]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
df = pd.read_csv('tips.csv')
X = df[['total_bill']]
y = df['tip']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

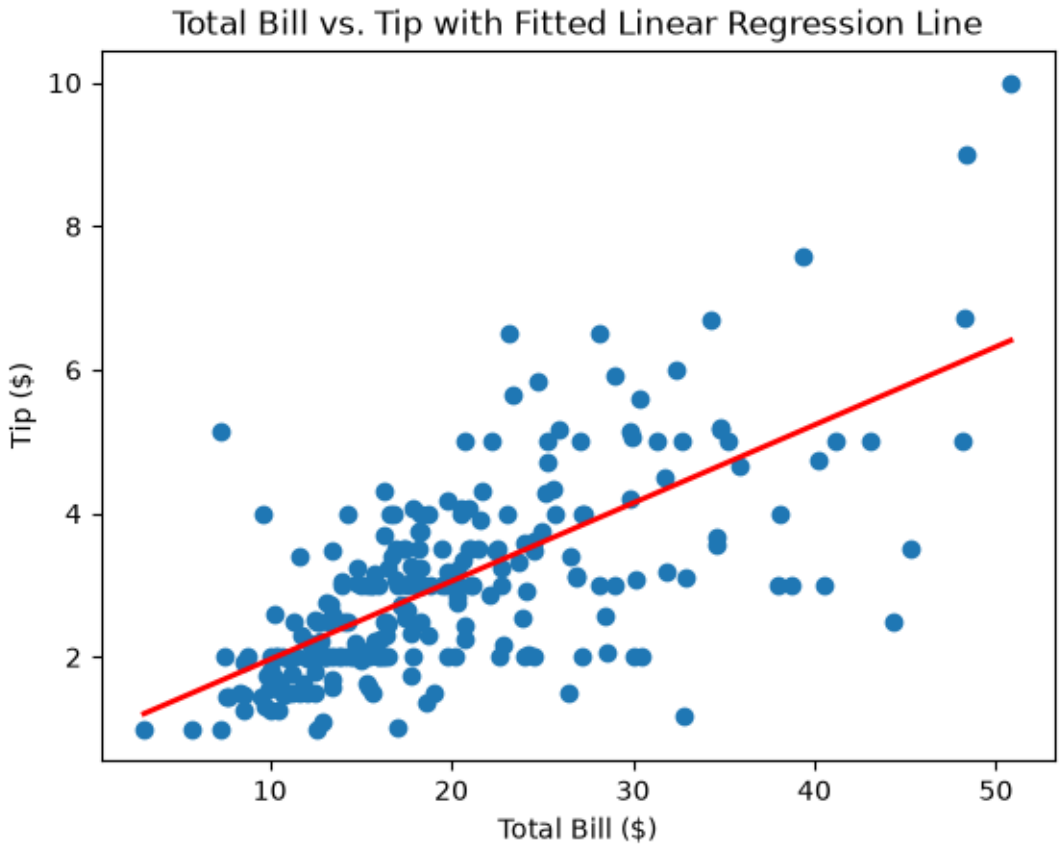
model = LinearRegression()
model.fit(X_train, y_train)

X_line = pd.DataFrame({'total_bill': sorted(df['total_bill'])})
y_pred = model.predict(X_line)

plt.figure
plt.scatter(df['total_bill'], df['tip'])
plt.plot(X_line, y_pred, color='red', linewidth=2, label=f'Fitted Line: y =␣
 ↪{model.coef_[0]:.2f}x + {model.intercept_:.2f}')

plt.title('Total Bill vs. Tip with Fitted Linear Regression Line')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.legend
plt.grid

plt.show()
```



[5]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris(as_frame=True)
df = iris.frame

X = df.drop(columns=['target'])
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (105, 4)
X_test shape: (45, 4)
y_train shape: (105,)
```

```
y_test shape: (45,)
```

[6]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris(as_frame=True)
df = iris.frame
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

test_accuracy = knn.score(X_test, y_test)

print(test_accuracy)
```

```
1.0
```

[7]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

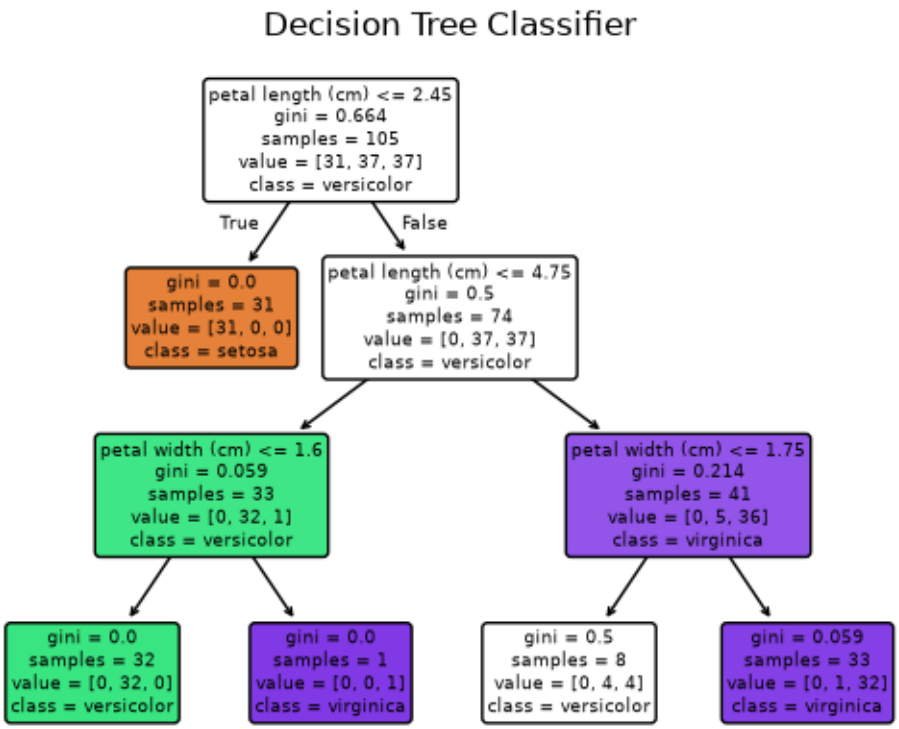
iris = load_iris(as_frame=True)
df = iris.frame
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

dt_classifier = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_classifier.fit(X_train, y_train)

test_accuracy = dt_classifier.score(X_test, y_test)
print(test_accuracy)

plt.figure
plot_tree(dt_classifier,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,
          rounded=True)
plt.title("Decision Tree Classifier")
plt.show()
```

```
1.0
```



Based on the decision tree, from the top, we ask, is the the petal length 2.45 centimeters or shorter? and down the decision tree decides if it's Versicolor or Virginica.

[8]:
```
iris = load_iris(as_frame=True)
df = iris.frame
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,␣
 ↪random_state=42)

tree_limited = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_limited.fit(X_train, y_train)
train_limited = tree_limited.score(X_train, y_train)
test_limited = tree_limited.score(X_test, y_test)

tree_full = DecisionTreeClassifier(random_state=42)
tree_full.fit(X_train, y_train)
train_full = tree_full.score(X_train, y_train)
test_full = tree_full.score(X_test, y_test)

print(tree_limited.score(X_test, y_test))
```

```
print(tree_full.score(X_test, y_test))
```

```
1.0
1.0
```

[ ]: